Blok B (80 min): Ollama i Python skripte za lokalni LLM

Cilj: svaki student ima Ollamu na svom računalu, mali model koji odgovara na pitanja, Python klijent za Ollama REST API, CLI alat `llm.py` i **5 pytest testova** koji prolaze. Rad se predaje kao pull request u osobni repozitorij `praktikum-<github-korisnik>`, istim tijekom kao u bloku A.

Kriterij za potpunost vježbe: `python -m pytest -v` pokazuje `5 passed`.

Sadržaj

1. Instalacija Ollame
2. Model i prvi razgovor
3. Ollama REST API iz terminala
4. Grana i Python okruženje
5. Klijent `ollama_client.py`
6. CLI `llm.py`
7. Pet pytest testova
8. Commit, push i pull request

> Hardver: model `llama3.2:1b` zauzima oko 1,3 GB na disku i treba oko 4 GB slobodnog RAM-a. Na slabijim računalima koristiti `qwen2.5:0.5b` (oko 400 MB). GPU nije obavezan; na CPU-u odgovor stiže za nekoliko sekundi.

1. Instalacija Ollame

WSL (Ubuntu) i Linux: Ollamu instaliramo unutar WSL-a, tamo gdje su i Python i repozitorij:

```
curl -fsSL https://ollama.com/install.sh | sh
ollama --version
```

Instalacijska skripta registrira systemd servis koji sluša na `127.0.0.1:11434`. Ako WSL nema systemd (`systemctl` javlja grešku), pokrenuti poslužitelj ručno u zasebnom terminalu:

```
ollama serve
```

macOS:

```
brew install ollama
brew services start ollama       # or download the app from https://ollama.com/download
```

Provjera:

```
curl http://localhost:11434      # "Ollama is running"
```

DevSecOps trenutak: Ollama nema autentifikaciju. Dok sluša na `127.0.0.1`, dostupna je samo s našeg računala. Postavljanje `OLLAMA_HOST=0.0.0.0` otvara je cijeloj mreži; to rješavamo tek s reverse proxyjem (tema 6).

2. Model i prvi razgovor

```
ollama pull llama3.2:1b          # or: ollama pull qwen2.5:0.5b
ollama list                      # installed models and their size
ollama run llama3.2:1b "Explain git in one sentence."
ollama run llama3.2:1b           # interactive, /bye to quit
ollama ps                        # loaded models, RAM usage, CPU or GPU
```

Prvi poziv je sporiji jer se model učitava u memoriju; sljedeći su brži dok je model učitan (zadano 5 minuta).

3. Ollama REST API iz terminala

CLI `ollama` je samo klijent za REST API. Isti pozivi koje ćemo raditi iz Pythona:

```
curl -s http://localhost:11434/api/tags | python3 -m json.tool

curl -s http://localhost:11434/api/generate -d '{
  "model": "llama3.2:1b",
  "prompt": "What is a pull request?",
  "stream": false
}' | python3 -m json.tool

curl -s http://localhost:11434/api/chat -d '{
  "model": "llama3.2:1b",
  "messages": [
    {"role": "system", "content": "Answer in one sentence."},
    {"role": "user", "content": "What is a merge commit?"}
  ],
  "stream": false
}' | python3 -m json.tool
```

Što pogledati u odgovoru: `response` (generate) ili `message.content` (chat), te `eval_count` i `eval_duration` (broj tokena i vrijeme u nanosekundama; tokeni po sekundi trebat će nam u temi 7). Bez `"stream": false` API vraća odgovor token po token, kao niz JSON redaka.

Endpoint	Namjena
`GET /`	radi li poslužitelj
`GET /api/tags`	popis lokalnih modela
`POST /api/generate`	jedan prompt, jedan odgovor
`POST /api/chat`	razgovor: lista poruka s ulogama `system`, `user`, `assistant`

4. Grana i Python okruženje

Radimo u osobnom repozitoriju iz tjedna 1, na novoj grani (konvencija iz bloka A):

```
cd ~/praktikum-$(gh api user -q .login)
git switch main && git pull
git switch -c feature/ollama-client
source .venv/bin/activate
pip install requests python-dotenv pytest
pip freeze > requirements.txt
```

Konfiguracija ide u `.env` (već je u `.gitignore` iz tjedna 1), a u repozitorij ide samo primjer:

```
cat > .env.example <<'EOF'
OLLAMA_BASE_URL=http://localhost:11434
OLLAMA_MODEL=llama3.2:1b
EOF
cp .env.example .env
```

Zašto `OLLAMA_BASE_URL`, a ne `OLLAMA_HOST`? `OLLAMA_HOST` već koristi sama Ollama (adresa na kojoj sluša, bez `http://`), pa bi se dvije namjene sudarile.

5. Klijent `ollama_client.py`

Sav HTTP je u jednoj datoteci; CLI i testovi koriste samo njezine funkcije. Spremiti kao `ollama_client.py`:

```
"""Minimal client for the local Ollama REST API
(https://github.com/ollama/ollama/blob/main/docs/api.md)."""
import os

import requests
from dotenv import load_dotenv

load_dotenv()

DEFAULT_BASE_URL = "http://localhost:11434"
DEFAULT_MODEL = "llama3.2:1b"
TIMEOUT_SECONDS = 120


class OllamaError(RuntimeError):
    """Raised when Ollama is unreachable or returns an error."""


def get_base_url() -> str:
    return os.getenv("OLLAMA_BASE_URL", DEFAULT_BASE_URL).rstrip("/")


def get_model() -> str:
    return os.getenv("OLLAMA_MODEL", DEFAULT_MODEL)


def _request(method: str, path: str, **kwargs) -> requests.Response:
    url = f"{get_base_url()}{path}"
    try:
        response = requests.request(method, url, timeout=TIMEOUT_SECONDS, **kwargs)
    except requests.ConnectionError as error:
        raise OllamaError(f"Ollama is not reachable at {get_base_url()} (start it with:
ollama serve)") from error
    if response.status_code != 200:
        raise OllamaError(f"Ollama returned HTTP {response.status_code}:
{response.text}")
    return response


def is_running() -> bool:
    try:
        _request("GET", "/")
    except OllamaError:
```

```
        return False
    return True


def list_models() -> list[str]:
    data = _request("GET", "/api/tags").json()
    return [model["name"] for model in data.get("models", [])]


def generate(prompt: str, model: str | None = None) -> str:
    payload = {"model": model or get_model(), "prompt": prompt, "stream": False}
    return _request("POST", "/api/generate", json=payload).json()["response"]


def build_messages(question: str, system: str | None = None, history: list[dict] | None
= None) -> list[dict]:
    messages = []
    if system:
        messages.append({"role": "system", "content": system})
    messages.extend(history or [])
    messages.append({"role": "user", "content": question})
    return messages


def chat(messages: list[dict], model: str | None = None) -> str:
    payload = {"model": model or get_model(), "messages": messages, "stream": False}
    return _request("POST", "/api/chat", json=payload).json()["message"]["content"]
```

Isprobati u interaktivnom Pythonu:

```
python -c "import ollama_client as o; print(o.is_running(), o.list_models())"
python -c "import ollama_client as o; print(o.generate('Say hello in Croatian.'))"
```

6. CLI `llm.py`

Isti argparse obrazac kao `greet.py` iz tjedna 1. Spremiti kao `llm.py`:

```
"""Command line interface for talking to a local Ollama model."""
import argparse
import sys

import ollama_client


def cmd_status(args: argparse.Namespace) -> int:
    if ollama_client.is_running():
        print(f"Ollama is running at {ollama_client.get_base_url()}")
        return 0
    print(f"Ollama is NOT running at {ollama_client.get_base_url()}", file=sys.stderr)
    return 1


def cmd_models(args: argparse.Namespace) -> int:
    for name in ollama_client.list_models():
        print(name)
    return 0
```

```
def cmd_ask(args: argparse.Namespace) -> int:
    messages = ollama_client.build_messages(args.question, system=args.system)
    print(ollama_client.chat(messages, model=args.model))
    return 0


def cmd_chat(args: argparse.Namespace) -> int:
    history: list[dict] = []
    print("Type your message, empty line to quit.")
    while True:
        question = input("> ").strip()
        if not question:
            return 0
        messages = ollama_client.build_messages(question, system=args.system,
history=history)
        answer = ollama_client.chat(messages, model=args.model)
        print(answer)
        history += [{"role": "user", "content": question}, {"role": "assistant",
"content": answer}]


def build_parser() -> argparse.ArgumentParser:
    parser = argparse.ArgumentParser(prog="llm", description="Talk to a local Ollama
model.")
    parser.add_argument("--model", default=None, help="model name (default: OLLAMA_MODEL
or llama3.2:1b)")
    parser.add_argument("--system", default=None, help="optional system prompt")
    subcommands = parser.add_subparsers(dest="command", required=True)

    subcommands.add_parser("status", help="check that Ollama is
running").set_defaults(handler=cmd_status)
    subcommands.add_parser("models", help="list locally installed
models").set_defaults(handler=cmd_models)

    ask = subcommands.add_parser("ask", help="ask one question")
    ask.add_argument("question", help="the question to ask")
    ask.set_defaults(handler=cmd_ask)

    subcommands.add_parser("chat", help="interactive chat with
history").set_defaults(handler=cmd_chat)
    return parser


def main() -> int:
    args = build_parser().parse_args()
    try:
        return args.handler(args)
    except ollama_client.OllamaError as error:
        print(f"ERROR: {error}", file=sys.stderr)
        return 1


if __name__ == "__main__":
    sys.exit(main())
```

Isprobati:

```
python llm.py --help
python llm.py status; echo $?
```

```
python llm.py models
python llm.py ask "What is a fast-forward merge?"
python llm.py --system "Answer like a pirate." ask "What is git?"
python llm.py chat                          # the model remembers earlier messages
sudo systemctl stop ollama                  # or stop "ollama serve" with Ctrl+C
python llm.py ask "Hello?"; echo $?         # ERROR: Ollama is not reachable ..., exit
code 1
sudo systemctl start ollama
```

Pitanje za grupu: kako model u `chat` načinu "pamti" prethodne poruke? (Ne pamti; svaki put šaljemo cijelu povijest u `messages`.)

7. Pet pytest testova

Testovi **ne smiju ovisiti o tome radi li Ollama**: CI runner (tema 9) nema Ollamu ni model. Zato `requests.request` zamijenimo lažnom funkcijom (pytest `monkeypatch`) koja vraća unaprijed pripremljen odgovor i pamti s čime je pozvana.

Napraviti `tests/test_ollama_client.py` s ovim početkom i prvim testom:

```
import pytest
import requests

import ollama_client


class FakeResponse:
    def __init__(self, data: dict, status_code: int = 200):
        self._data = data
        self.status_code = status_code
        self.text = str(data)

    def json(self) -> dict:
        return self._data


@pytest.fixture
def fake_request(monkeypatch):
    """Replaces requests.request and records every call it receives."""
    calls = []

    def install(response: FakeResponse | None = None, error: Exception | None = None):
        def fake(method, url, **kwargs):
            calls.append({"method": method, "url": url, **kwargs})
            if error:
                raise error
            return response

        monkeypatch.setattr(ollama_client.requests, "request", fake)
        return calls

    return install


def test_build_messages_puts_system_prompt_first():
    messages = ollama_client.build_messages("What is git?", system="Answer briefly.")

    assert messages == [
        {"role": "system", "content": "Answer briefly."},
```

```
        {"role": "user", "content": "What is git?"},
    ]
```

Zatim samostalno napisati još četiri testa:

#	Test	Što provjerava
1	`test_build_messages_puts_system_prompt_first`	(gore) system poruka je prva, pitanje zadnje
2	`test_list_models_returns_model_names`	za lažni odgovor `{"models": [{"name": "llama3.2:1b"}, {"name": "qwen2.5:0.5b"}]}` funkcija vraća samo imena
3	`test_generate_sends_model_and_prompt_without_streaming`	`generate` šalje na `/api/generate` točno `{"model": ..., "prompt": ..., "stream": false}`; model postaviti s `monkeypatch.setenv("OLLAMA_MODEL", "test-model")` i provjeriti `calls[0]["json"]`
4	`test_chat_returns_assistant_message_content`	iz odgovora `{"message": {"role": "assistant", "content": "..."}}` vraća samo tekst
5	`test_generate_raises_ollama_error_when_server_is_down`	kad `requests` baci `requests.ConnectionError`, klijent baca `OllamaError` (`pytest.raises(..., match="not reachable")`)

Pokretanje (iz korijena repozitorija, da `import ollama_client` radi):

```
python -m pytest -v
```

Očekivano: `5 passed`. Svaki test slijedi obrazac pripremi → pozovi → provjeri (arrange, act, assert). Provjeriti da test stvarno nešto testira: privremeno pokvariti `build_messages` (npr. zamijeniti redoslijed) i vidjeti da test 1 padne.

Dodatni zadatak: šesti, integracijski test koji zove pravu Ollamu i preskače se kad ona ne radi:

```
@pytest.mark.skipif(not ollama_client.is_running(), reason="Ollama is not running")
def test_real_model_answers():
    assert ollama_client.generate("Reply with the single word: ok").strip() != ""
```

8. Commit, push i pull request

```
python -m pytest -v | tee pytest.txt
git add ollama_client.py llm.py tests/ requirements.txt .env.example pytest.txt
git status                        # .env and .venv must NOT be listed
git commit -m "feat: add Ollama client, CLI and tests"
git push -u origin HEAD
gh pr create --base main --title "feat: add Ollama client, CLI and tests" --body "Week
2, block B"
```

PR spojiti sam (osobni repozitorij) strategijom kolegija: fast-forward kad je moguć, inače merge commit. To radi `git merge` bez opcija; GitHub PR sam označi kao *merged*:

```
git switch main && git pull
git merge feature/ollama-client     # here main has not moved, so this is a fast-forward
git push origin main
```

Ako je release-please iz tjedna 1 aktivan, `feat:` commit na `main` otvara release PR za `0.2.0`.

Predaja (praktični zadatak 2)

Spojen PR u `praktikum-<github-korisnik>` s datotekama `ollama_client.py`, `llm.py`, `tests/test_ollama_client.py` (5 testova), `requirements.txt`, `.env.example` i `pytest.txt` u kojem piše `5 passed`.

Česti problemi

Simptom	Rješenje
`curl: (7) Failed to connect to localhost port 11434`	Ollama ne radi: `sudo systemctl start ollama` ili `ollama serve` u zasebnom terminalu
`Error: model "llama3.2:1b" not found` / HTTP 404	Model nije preuzet: `ollama pull llama3.2:1b`, ili `OLLAMA_MODEL` u `.env` ne odgovara `ollama list`
Odgovor traje minutama, računalo se zamrzava	Premalo RAM-a: `qwen2.5:0.5b`, zatvoriti druge programe; provjeriti `ollama ps`
`ModuleNotFoundError: No module named 'ollama_client'`	Pokretati `python -m pytest` iz korijena repozitorija, ne `pytest` iz `tests/`
`ModuleNotFoundError: No module named 'requests'`	venv nije aktivan: `source .venv/bin/activate`
`TypeError: unsupported operand type(s) for |`	Python stariji od 3.10; provjeriti `python --version` (tjedan 1 traži 3.10+)
Testovi prolaze samo kad Ollama radi	Test zove pravi API: zamijeniti `requests.request` preko `fake_request` fixturea